

Sommer 24

Del I

Konstanter

1. [5]

På hvilke måter (en eller flere) kan man deklarere en konstant i C++?

- ☐ Skrive variabelnavnet med blokk-bokstaver.
- ✓ **Bruke nøkkelordet `const`.**
- ✓ **Bruke nøkkelordet `constexpr`.**
- ☐ Bruke nøkkelordet `constant`.

Funksjoner

2. [5]

I hvilke (en eller flere) av situasjonene under burde man bruke *pass-by-reference*?

- ☐ Når man gir en heltallsvariabel til en funksjon som regner ut kvadratroten og returnerer resultatet.
- ✓ **Når man gir et stort array av strenger til en funksjon som gjør alle forbokstaver store.**
- ✓ **Når man gir en stor vektor av flyttall til en funksjon som regner ut gjennomsnittet av alle tallene i vektoren og returnerer gjennomsnittet.**
- ✓ **Når man har en funksjon som tar inn en `unique_ptr`, men hvor funksjonen ikke endrer på verdien den peker til.**

Klasser

3. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ✓ **En kopi-konstruktør kan brukes til å lage et nytt klasseobjekt ved å kopiere verdier fra et annet klasseobjekt av samme klasse.**
- ☐ Enhver klasse har en kopi-konstruktør.
- ✓ **En kopi-konstruktør kan defineres slik at den initialiserer et nytt klasseobjekt ved å kopiere verdier fra et annet klasseobjekt av en hvilken som helst klasse.**
- ☐ En kopi-konstruktør kan kun brukes til grunn kopiering (*shallow copy*).

enum-klasser

4. [5] Se på koden under.

```
1  enum class Color {
2      red = 2, blue, yellow
3  };
4
5  Color c = Color::yellow;
```

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☐ Det er ikke mulig å definere en ny enum-klasse med verdiene `red`, `blue`, og `yellow`, fordi de allerede finnes i `Color`.
- ☐ Verdiene til enumeratorene `red`, `blue` og `yellow` lagres som strenger.

- ✓ **Det er ikke mulig å direkte sammenligne en variabel av typen Color med en int.**
- Det er ikke mulig å tilordne verdien `Color::yellow` til en variabel fordi det er siste verdien som er definert i enum class.

Tilgangsnivå

5. [5]

Se på koden under.

```
class B {
    int y = 5;
protected:
    void f() const {}
    void g() {}
public:
    std::string s = "hello";

    virtual void c() = 0;
    void d() {}
};

class A : public B {
    int x;
public:
    void a() {}
    void b() {}
};
```

Hvor mange medlemsfunksjoner har klassen A? Du trenger ikke telle med funksjoner som er standard-definert.

- ✓ **5**
- 2
- 4
- 6

RAII

6. [5]

Se på klassedeklarasjonen under.

```
class A {
    std::unique_ptr<std::ofstream> fileStream;

public:
    A(const string &filename) : fileStream(new std::ofstream(filename)) {}

    }
};
```

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ✓ **Klassen overholder RAI (Resource Acquisition Is Initialization).**
- ✓ **Klassen har en implisitt standard (default) destruktør.**

- ☐ Klassen mangler en destruktør som lukker filstrømmen `fileStream` for at den skal overholde RAII (*Resource Acquisition Is Initialization*).
- ☐ Når en instans av klassen A går ut av skop, oppstår det en minnelekkasje.

Beholdere

7. [5]

Hvilke (en eller flere) av de følgende beholderene har ikke elementene liggende rett etter hverandre i minnet?

- ☒ `std::map`
- ☐ `std::vector`
- ☒ `std::set`
- ☐ `std::array`

Lagring

8. [5]

Hvor lagres de ulike programelementene? (Kryss av i riktig boks.)

	Begge	Heap/Free store	Stack
Medlemsvariabler	x		
Lokale variabler			x
Funksjonsargumenter			x
Et objekt allokert med <code>new</code> -operatoren		x	

`new`-operatoren

9. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☒ `new`-operatoren returnerer en peker til et objekt i minnet av en bestemt type.
- ☐ `new`-operatoren allokterer alltid på *stack-lageret* (*stack storage*).
- ☐ Man må bestemme hvor mange objekter av en type som skal allokeres før kjøretid når man bruker `new`-operatoren.
- ☐ Uttrykket `int* a = new int{5};` vil allokere et array av 5 `int` på heap.

Templates

10. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☒ **Kompilatoren kan i noen tilfeller løse template-parametre uten at typen til parametrene er oppgitt.**
- ☐ Klassesemplates kan ikke ha standard (*default*) template-parametre.
- ☐ Template-argumenter kan kun være klasser.
- ☐ Templates må plasseres i en header-fil. Templates må plasserast i ei header-fil.

I/O

11. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ✓ **Operativsystemet vil opprette en fil man prøver å skrive til hvis den ikke eksisterer.**
- ✓ **En fil er en sekvens med bytes.**
- ☐ Operativsystemet vil opprette en fil man prøver å lese fra hvis den ikke eksisterer.
- ☐ Det er ikke mulig å skrive private klassemedlemmer til fil.

Unntakshåndtering

12. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☐ Minne som har blitt allokert med new-operatoren i funksjonen blir automatisk frigjort når et unntak blir kastet.
- ✓ **Flere catch-blokker kan behandle samme type unntak.**
- ☐ Dersom det ikke finnes en catch-blokk for et utløst unntak, vil unntaket ignoreres og programmet fortsette normal kjøring fra punktet der unntaket ble utløst.
- ✓ **Programmet avslutter dersom et unntak ikke blir håndtert.**

Del II

Bibliotek og linking

13. [5]

Forklar kort hva et bibliotek er og hvorfor linking er nyttig når man bruker bibliotek.

Solution: Et bibliotek er kode man har tilgang til gjennom deklarasjoner i en inkludert fil. Linkeren kombinerer delene av programmet man skriver selv og bibliotekene man bruker til en enkel kjørefil (*executable*).

(2.4, s.51 i læreboka)

Funksjoner

14. [5] Forklar kort forskjellen på en funksjonsdeklarasjon og en funksjonsdefinisjon.

Solution:

Funksjonsdeklarasjon Spesifiserer navnet, parameterne og typen til returverdien til funksjonen. Forteller kompilatoren at funksjonen eksisterer slik at den kan kalles andre steder i programmet.

Funksjonsdefinisjon Inkluderer en funksjonsdeklarasjon, men også implementasjonen til funksjonen.

(4.5, s. 113-117 i læreboka, Forelesning 2)

Typekonvertering

15. [5]

Forklar kort forskjellen på implisitt og eksplisitt typekonvertering.

Solution: Implisitt typekonvertering gjøres av kompilatoren og fungerer dersom typen man konverterer til bruker flere bits enn typen man konverterer fra. Eksplisitt typekonvertering gjøres av programmereren og vil fungere selv hvis typekonverteringen fører til tap av presisjon. For å gjøre eksplisitt typekonvertering kan man bruke f.eks. `static_cast`. (Eksplisitt typekonvertering gjør det også tydelig at konverteringen er intensjonell.)

(Forelesning 2)

Kopiering

16. [5]

Forklar kort forskjellen på grunn kopiering (*shallow copy*) og dyp kopiering (*deep copy*).

Solution:

Grunn kopiering Kopierer kun pekeren slik at originalen og kopien refererer til samme objekt.

Dyp kopiering Kopierer det en peker peker til slik at originalen og kopien peker til distinkte objekt.

(18.3.3, s.636-637 i læreboka)

Beholdere og kopiering

17. [5]

Hvorfor er det unødvendig å bruke `std::unique_copy()` for å kopiere elementene i en `std::vector` til et `std::set`?

Solution: En `std::vector` kan inneholde duplikate verdier, men duplikate verdier ignoreres når de plasseres i et `std::set` og `std::unique_copy()` kopierer unike verdier, så det vil være overflødig.

(21.2, s.760 i læreboka)

Beholdere og funksjoner

18. [5]

Forklar kort hva som skjer når man kaller `push_back()` på en `std::vector`.

Solution: Når man kaller `push_back()` funksjonen på en `std::vector` settes et nytt element på slutten av vektoren, men først må minnekapasiteten til vektoren sjekkes. Hvis det ikke er nok plass til det nye elementet i minnet som er allokert til vektoren, allokteres mer minne til vektoren (ofte doubles kapasiteten) før det nye elementet settes inn. Størrelsen til vektoren (`size`) øker med én, mens kapasiteten (`capacity`) reflekterer størrelsen til minnet allokert.

(4.2.6, s. 119-120, 19.2.4, s. 674-675 i læreboka, forelesning 13)

Pekere

19. [5]

Se på de to kodesnuttene under.

Kodesnutt 1:

```
1 int* p = new int{10};
2 delete p;
3 delete p;
```

Kodesnutt 2:

```
1 int* p = new int{10};
2 p = new int{10};
3 delete p;
```

Kodesnutt 1 vil føre til en kjøretidsfeil, men det vil ikke kodesnutt 2. Forklar kort hvorfor.

Solution: Kodesnutt 1 er et eksempel på dobbel frigjøring (*double free*), hvor vi prøver å frigjøre minne som allerede er frigjort. I kodelinje 3 eier vi ikke lenger minnet som pekeren p pekte til, og vi får en kjøretidsfeil når vi prøver å frigjøre minnet på nytt.

I kodesnutt 2 vil vi få en minnelekkasje siden det allokeres nytt minne til pekeren p i linje 2 uten at minnet som ble allokert til p i linje 1 har blitt frigjort først.

(17.4, s.598-600 i læreboka)

Klasser

20. [5] Forklar kort hvilke problemer det er i kodesnutten under.

```
1 class Counter {
2 public:
3     void count(const int b) const {
4         a += b;
5         std::cout << "The counter is now set to: " << a << std::endl;
6     }
7
8 private:
9     int a = 0;
10 };
11
12 int main(void) {
13     int b = 1;
14     Counter a;
15     a.count(b);
16     return EXIT_SUCCESS;
17 }
```

Solution: Medlemsfunksjonen count() i Counter-klassen er deklarert som konstant, men den prøver å endre medlemsvariabelen a. En konstant (const) medlemsfunksjon kan ikke endre tilstanden til objektet den kalles på.

(9.7.4, s. 330-332 i læreboka)

21. [10] Se på koden under. Sjå på koden under.

```
1  #include <vector>
2
3  using namespace std;
4
5  class A {
6  private:
7      int x;
8  public:
9      A(int x_) : x(x_) {}
10 };
11
12 int main(void) {
13     unique_ptr<A> a {new A(5)};
14     vector<unique_ptr<A>> vec;
15     vec.push_back(a);
16
17     return 0;
18 }
```

(a) (5 points) Hvorfor kompilerer ikke koden? Forklar kort.

Solution: Problemet er at det ikke er mulig å legge den unike pekeren `a` i vektoren `vec` med `push_back`, da dette vil innebære kopiering av en unik peker, noe som ikke er tillatt. (19.5.4, s.703-705, forelesning 11)

(b) (5 points) Hva ville du gjort for å løse problemet?

Solution: Her er det flere alternativer (som alle godtas). Bytte ut `unique_ptr` med `shared_ptr`, bruke `vec.push_back(make_unique<A>(5))` for å opprette og legge til den unike pekeren i vektoren, eller bruke `vec.push_back(move(a))` for å flytte den unike pekeren. (19.5.4, s.703-705, forelesning 11)

Templates

22. [10]

Se på koden under.

```
1  #include <iostream>
2
3  template<typename T>
4  struct Point {
5      T x = 0;
6      T y = 0;
7  };
8
9  int main(void) {
10     Point<int> p1{2,4};
11     Point<int> p2{1,6};
12
13     if (p1 > p2) {
14         std::cout << "The first point is largest" << std::endl;
```

```

15     } else if (p1 == p2) {
16         std::cout << "The points are equal" << std::endl;
17     } else {
18         std::cout << "The second point is largest" << std::endl;
19     }
20
21     return 0;
22 }

```

(a) (5 points) Hvorfor kompilerer ikke koden? Forklar kort.

Solution: Operatorene `>`, `==` og `<` er ikke definert for `Point`.
(9.6, s.321-322 i læreboka, forelesning 9 og 13, øving 11)

(b) (5 points) Hva må gjøres for å løse problemet gitt at du ikke kan endre noe i `main`? Du trenger ikke skrive kode, men forklar kort hva du vil gjøre.

Solution: Man må overlaste operatorene `>`, `==` og `<` for `Point`.
(9.6, s.321-322 i læreboka, forelesning 9, øving 11)

Del III: Skiskyting og datastrukturer (180p)

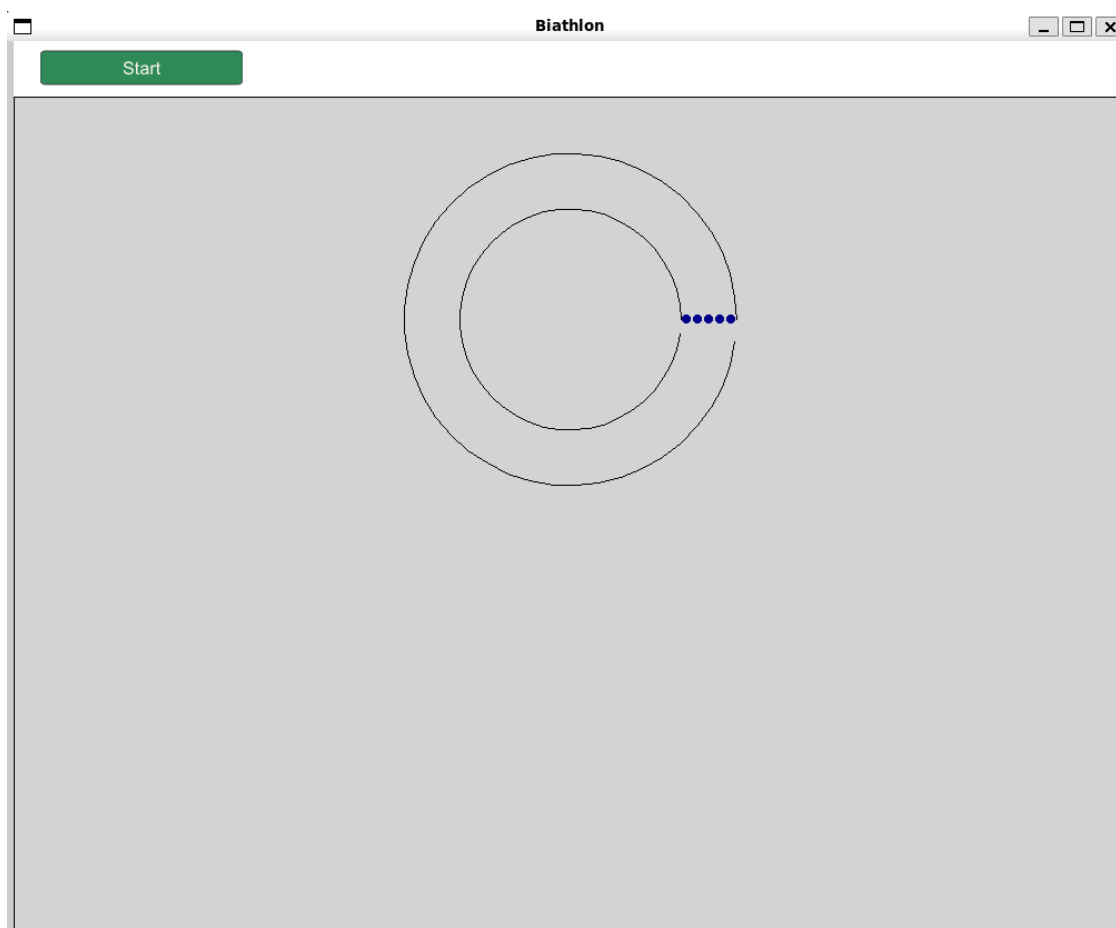
Introduksjon

Del 3 er delt inn i to hovedbolker:

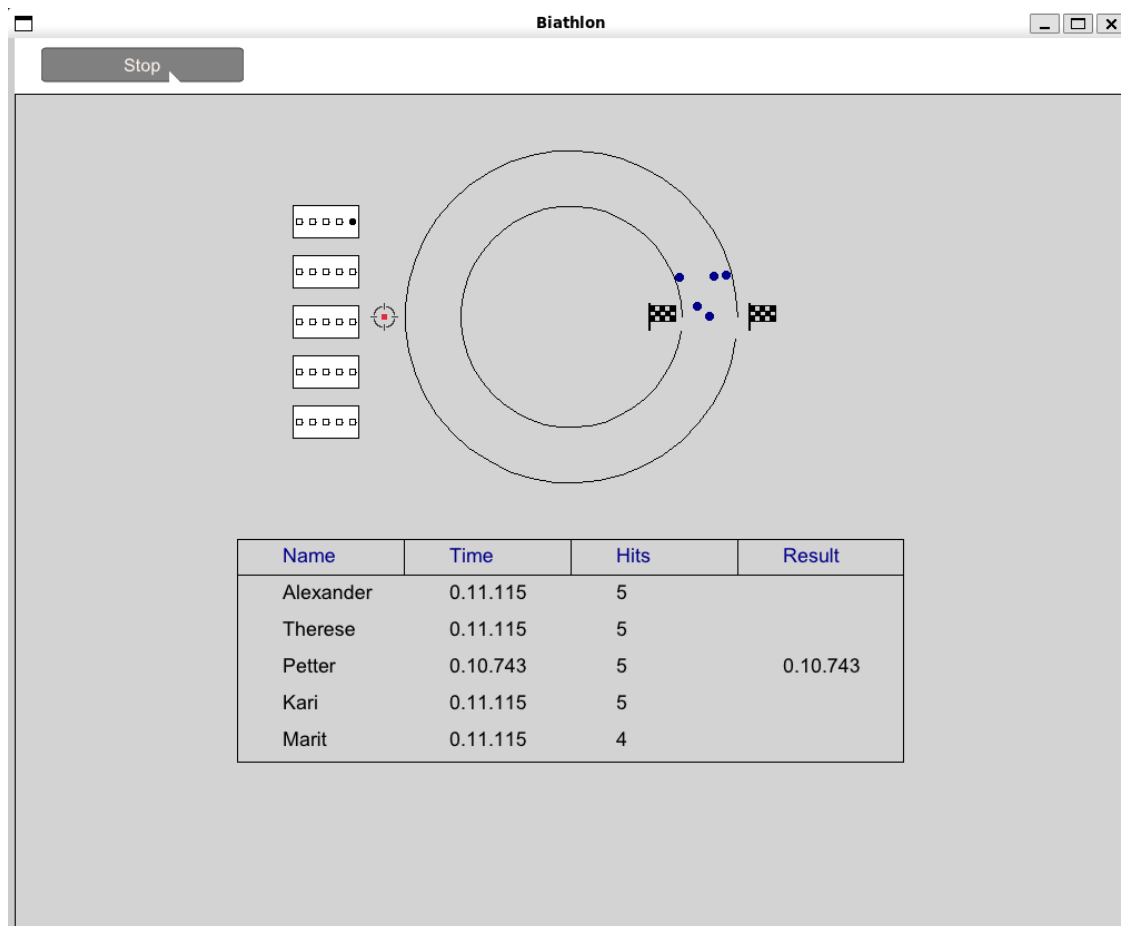
- En skiskytter-simulering som produserer data.
- En tabellstruktur (tenk regneark som i Excel) som viser frem dataen og holder oversikt over resultatene.

Figur 1 viser hvordan kjøring av den utdelte koden ser ut. **Figur 2** viser hvordan kjøring av koden skal se ut etter at alle oppgavene er løst.

Simulasjonen kontrolleres av et Simulator-objekt som har en liste med alle Skier-objektene. I tillegg har Simulator en unique_ptr til et Stopwatch-objekt som registrerer tiden, og et Table-objekt for resultatene.



Figur 1: Skjerm bilde av kjøring av den utdelte koden. I animasjonsvinduet ser man en funksjonsbar bestående av en Start/Stopp-knapp og to sirkler som skal illustrere en skiløype.



Figur 2: Skjerm bilde av kjøring av koden når alle oppgavene er løst.

Tabellen er generell, slik at hver celle i teorien kan inneholde en vilkårlig datatype, men tabellen er initialisert med `std::string` i simulasjonen. Et `Table`-objekt lagrer verdiene i en to-dimensjonal liste kalt `data`. Hver celle i tabellen består av en `TableData`-struktur. For å holde kontroll på hvilke kolonner som finnes i tabellen, lagres i tillegg kolonnenavnene i listen `columnTitles`. Denne listen inneholder alle de ulike strengene man kan finne i feltene `TableData::columnTitle` i `data`. Under kan du se definisjoner for `TableData`-strukturen og deler av `Table`-klassen.

```
template <typename T>
struct TableData
{
    string columnTitle;
    T value;
};

class Table
{
public:
    void draw(AnimationWindow &window, int height, int width, Point topLeftCorner);
    void storeTable(filesystem::path filepath);
```

```
// (...)
// more functionality below for
// adding and updating values

private:
    // (...)

    vector<string> columnTitles;
    vector<vector<TableData<string>>> data;
};
```

Fordelen med denne måten å lagre cellene på er at radene i data kun inneholder en kolonne dersom den har en verdi. Det betyr at dersom det blir lagt til en ny kolonne, trenger vi ikke gå gjennom hele tabellen og legge til NULL-verdier for radene som allerede eksisterer. Sagt på en annen måte, lagrer vi ikke NULL-verdier hvis de ikke eksplisitt blir lagt inn.

I [Figur 3](#), [Figur 4](#) og [Figur 5](#) kan du se et eksempel på hvordan dette kan henge sammen.

	A	B	C	D
1	Navn	Studie	Retning	Bosted
2	Henrik	Sykepleie		Bergen
3	Ola			Trondheim
4	Kari	Psykologi		
5	Anne	Veterinær	Smådyr	Ås

Figur 3: Eksempel på en tilfeldig tabell. Noter deg at ikke alle feltene er fylt inn.

```
[0]    [1]    [2]    [3]
{"Navn", "Studie", "Retning", "Bosted"}
```

Figur 4: Hvordan listen columnTitles kan se ut for tabellen i [Figur 3](#). Tallene over listen skal vise indexen til de ulike verdiene.

```
{
    [0]          [1]          [2]          [3]
    {"Navn": "Henrik", "Studie": "Sykepleie", "Bosted": "Bergen"},
    {"Navn": "Ola", "Bosted": "Trondheim"},
    {"Navn": "Kari", "Studie": "Psykologi"},
    {"Navn": "Kari", "Studie": "Veterinær", "Retning": "Smådyr", "Bosted": "Ås"}
}
```

Figur 5: Hvordan den 2-dimensjonale listen data kan se ut for tabellen i [Figur 3](#). Tallene over listen skal vise indeksen til de ulike verdiene. Noter deg at ikke alle vektorene er like lange. Noter deg også at alle radene er sortert i samme rekkefølge som columnTitles. Det vil si at 'Navn' alltid kommer først og 'Bosted' alltid kommer sist.

Din oppgave er å utvide og legge til funksjonalitet i den utdelte koden. I .zip-filen finner du et kodeskjelett med tydelig markerte oppgaver.

Hvordan besvare oppgavene

Hvis du synes noen av oppgavene er uklare kan du oppgi hvordan du tolker dem og de antagelsene du gjør for å løse oppgaven som kommentarer i koden du leverer.

Oppgaveteksten

Oppgavetekstene vil ha følgende struktur:

Første del inneholder motivasjon, bakgrunnsinformasjon og hvordan koden er satt sammen for oppgaven.

Neste del er en tekstboks som inneholder oppgaven og spesifikke krav.

Siste del forklarer resultatet du kan forvente når du har fullført oppgaven.

Oppgavene

1. (5 points) T1: Start simulasjonen

Start-knappen trenger tilgang til variablene `running` og `watch` i `Simulator`-klassen for å kunne kjøre simulasjonen. Vi ønsker derimot ikke å gjøre variablene offentlige, hverken ved å gjøre dem `public` eller gjennom `get`-funksjoner. Din oppgave er å finne en annen måte å gi `ToggleButton`-klassen tilgang til disse variablene.

I `simulator.h` filen:

- Fjern eller kommenter ut kodelinjen
`#define START_BUTTON_NO_ACCESS_TO_RUNNING_AND_WATCH`
i `simulator.h`.
- Gi `ToggleButton`-klassen tilgang til variablene `running` og `watch` uten å endre tilgangsnivået fra `private`.

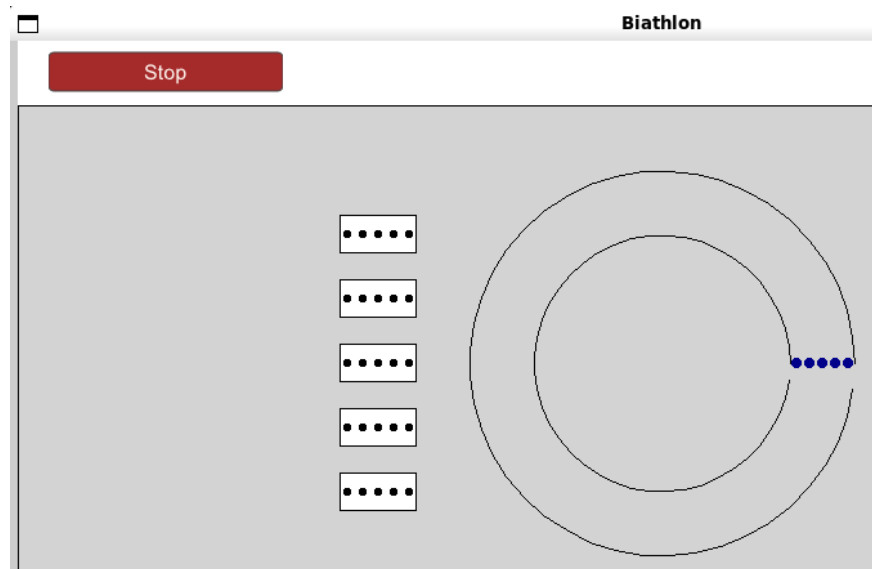
Tips 1: Du kan legge til kodelinjen `#define START_BUTTON_NO_ACCESS_TO_RUNNING_AND_WATCH` igjen dersom du ikke har løst oppgaven enda og ønsker å fjerne feilmeldingene du får fra kompilatoren.

Tips 2: Dersom du ikke får til denne oppgaven kan du kommentere ut eller fjerne linje 155 `simulator.h`. Dette vil gi alt i `Simulator` tilgangsnivået `public`, og du kan fortsette på de neste oppgavene uten ulemper.

```
// Remove this part to make everything public if you
// did not manage to complete task T1
// REMOVE
private:
// REMOVE
```

Når oppgave T1 er ferdig skal simuleringen starte når man trykker på Start-knappen. Dette ser man ved at knappen bytter mellom Start-knapp og Stop-knapp når den blir trykket på. I tillegg vil man

etter ca. 5 sekunder se at målskivene til utøverne dukker opp når de ankommer skytebanen som vist i Figur 6.



Figur 6: Simulasjonen etter at T1 er implementert og man har trykket på Start-knappen og ventet noen sekunder.

Solution:

```
// TASK: T1
// Give the ToggleButton class access to
// private members of the Simulator class.
// BEGIN: T1
    friend ToggleButton;
// END: T1
```

2. (5 points) **T2: Oppdater antall treff**

Når man nå trykker på Start-knappen vil simulasjonen kjøre, men målskivene (se Figur 6) blir ikke oppdatert når en utøver treffer. Det er fordi hit-funksjonen i Target ikke er implementert. Denne funksjonen skal øke hits-variabelen i Target med 1 hver gang den blir kalt. Du finner definisjonen til Target-klassen i simulator.h, som ser slik ut:

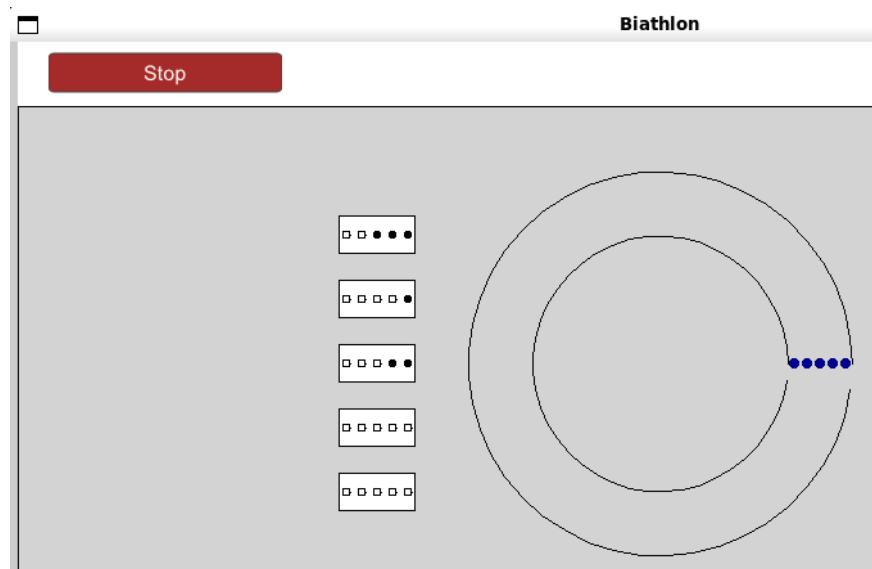
```
class Target
{
public:
    void hit();
    void draw(AnimationWindow *window, Point position, int size);
    int getHits();
};
```

```
private:
    int hits = 0;
};
```

Implementer funksjonen `Target::hit()` i `simulator.cpp`.

- `Target::hit()` skal øke `Target::hits`-variabelen med 1.

Når oppgave T2 er implementert vil man kunne se at målskivene blir oppdatert etter hvert som utøverne skyter og treffer som vist i **Figur 7**.



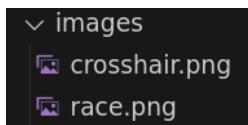
Figur 7: Målskivene vil bli oppdatert når en utøver treffer etter at oppgave T2 er implementert.

Solution:

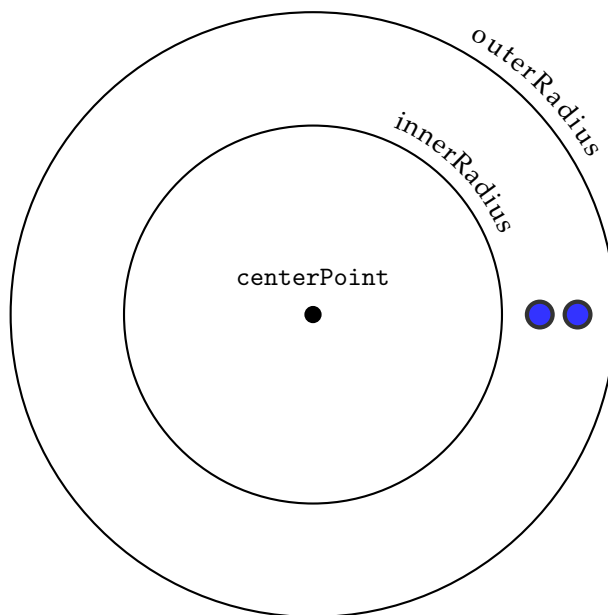
```
// TASK: T2
// Increment the local variable hits.
void Target::hit()
{
    // BEGIN: T2
    hits++;
    // END: T2
}
```

3. (10 points) T3: Marker kartet

Vi ønsker å markere start, mål og skyteområde på kartet i simulasjonen. I `images`-mappen i den utdelte koden finner du to bilder (`.png`-filer). Filen `crosshair.png` viser et trådkors og filen `race.png` viser et flagg.



Vi skal nå implementere funksjonen `Simulator::placeImages` som skal bruke bildene til å markere kartet. Denne funksjonen tar 4 parametre: `centerPoint`, `innerRadius`, `outerRadius` og `size`. `size` beskriver størrelsen som skal brukes på bildene. De andre variablene er illustrert i [Figur 8](#), og skal brukes til å plassere bildene på riktig sted.



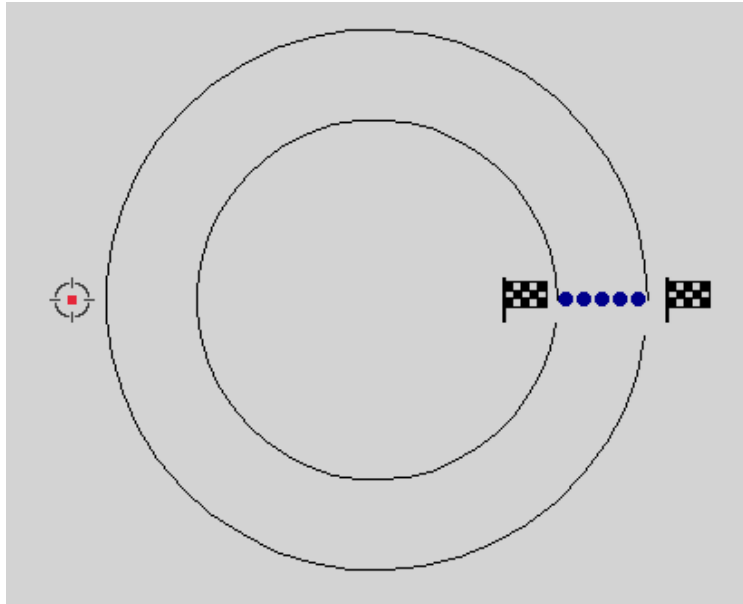
Figur 8: `centerPoint` er midtpunktet vi orienterer fra. `innerRadius` er avstanden fra `centerPoint` til den innerste sirkelen. `outerRadius` er avstanden fra `centerPoint` til den ytterste sirkelen.

Implementer funksjonen `Simulator::placeImages` i `simulator.cpp`.

- Bruk parametrene `centerPoint`, `innerRadius` og `outerRadius` til å tegne flagg ved start og mål, og et trådkors ved skytområdet.
- Bildefilene (.png-filene) som skal brukes ligger i `images`-mappen. Filen `crosshair.png` viser et trådkors og filen `race.png` viser et flagg.
- `size`-parameteren skal brukes til å sette riktig størrelse på bildene.

Hint: Man kan finne informasjon om hvordan man bruker bilder i dokumentasjonen til `AnimationWindow`.

Når oppgave T3 er gjort skal kartet se ut omtrent som kartet i [Figur 9](#). Det tar litt lenger tid før målskivene vises etter denne oppgaven er implementert.



Figur 9: Skjerm bilde av simulasjonen etter at oppgave T3 er implementert, med flagg ved start og mål, og trådkors ved skyteområdet.

Solution:

```
// TASK: T3
// Use the local window variable in Simulator
// and place two flags and one crosshair at the
// given locations. The .png files can be found in
// the images folder.
void Simulator::placeImages(TDT4102::Point centerPoint,
                           int innerRadius, int outerRadius, int size)
{
    // BEGIN: T3
    TDT4102::Point crosshairPosition =
    {
        centerPoint.x - outerRadius - 30,
        centerPoint.y - 12
    };
    TDT4102::Point flagPosLeft =
    {
        centerPoint.x + innerRadius - 30,
        centerPoint.y - 12
    };
    TDT4102::Point flagPosRight =
    {
        centerPoint.x + outerRadius + 10,
        centerPoint.y - 12
    };
};
```



```

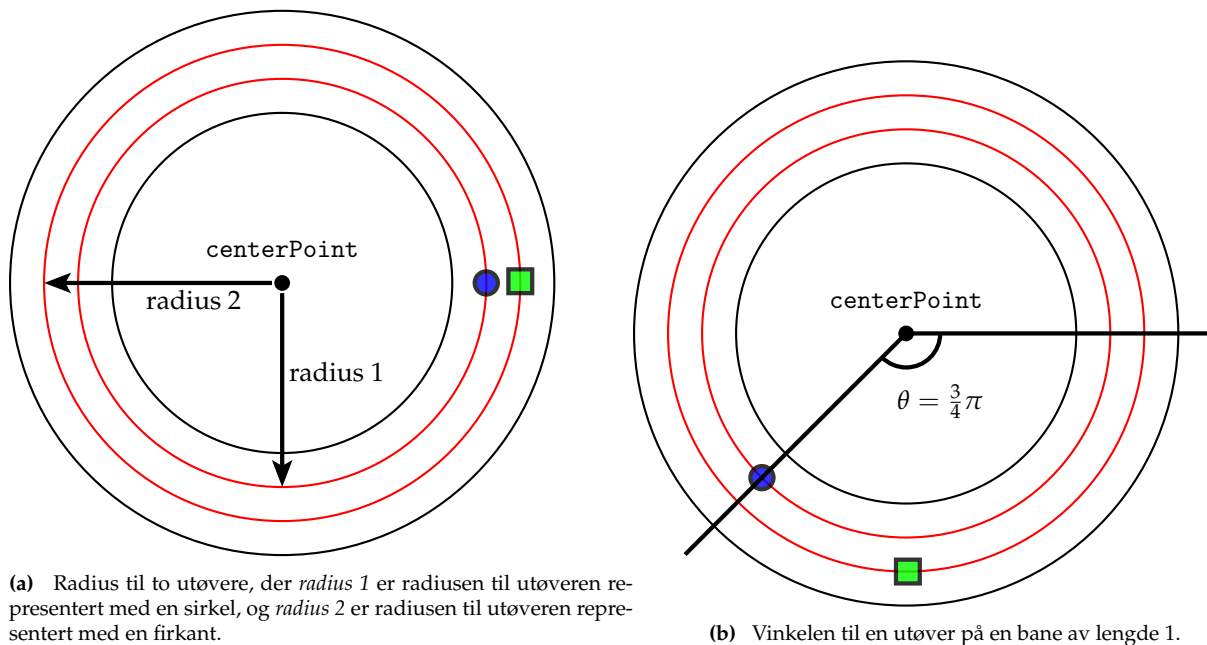
Image crosshair("images/crosshair.png");
Image flag("images/race.png");

window.draw_image(crosshairPosition, crosshair, size, size);
window.draw_image(flagPosLeft, flag, size, size);
window.draw_image(flagPosRight, flag, size, size);
// END: T3
}

```

4. (20 points) T4: Få utøverne ut i løypen

Vi ønsker å kunne visualisere progresjonen til hver enkelt utøver. Siden kartet er gitt som en sirkel, vil posisjonen til en utøver være gitt av en *radius* som forteller hvilken fil de ligger i og en *vinkel* som forteller hvor langt de har kommet. Dette er vist i henholdsvis **Figur 10a** og **Figur 10b**.



Figur 10: Oversikt over løypevariabler.

x -posisjonen til en utøver er dermed gitt av

$$x = x_c + r \cdot \cos \theta,$$

mens y -posisjonen til en utøver er gitt av

$$y = y_c + r \cdot \sin \theta,$$

hvor x_c og y_c representerer `centerPoint` sin x - og y -koordinat, og r er radiusen. θ er progresjonen til utøveren og er gitt av

$$\theta = 2\pi \cdot \frac{\text{position}}{\text{distance}},$$

hvor *distance* er lengden på banen og *position* er posisjonen til utøveren. Disse verdiene finner man henholdsvis i `Map::distance` og ved å bruke `Skier::getPosition()`. `Map`-strukturen er definert i `simulator.h` og er lagret som en variabel i `Simulator`-klassen. `Skier::getPosition` er implementert i `simulator.cpp`.

```
struct Map
{
    double distance;
    double shootingPosition;
};

double Skier::getPosition() { return position; }
```

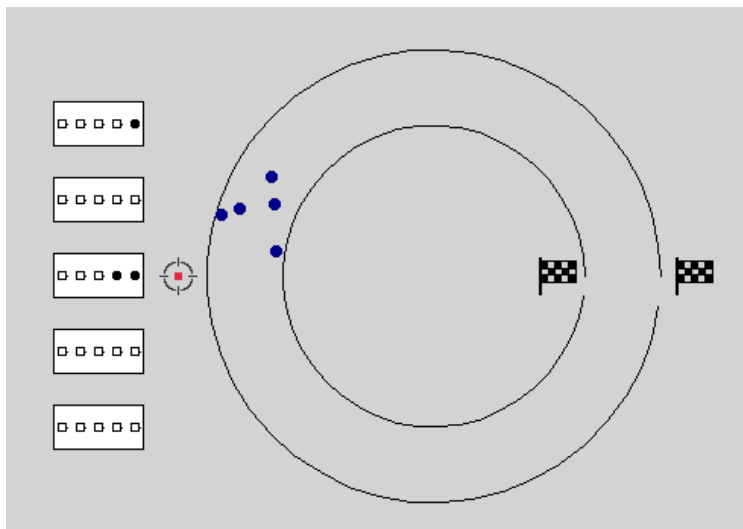
Implementer funksjonen `Simulator::calculateSkierPosition` i `simulator.cpp`.

- Funksjonen skal returnere et `TDT4102::Point` som illustrerer hvor i løypen utøveren befinner seg.
- x -posisjonen til en utøver er gitt av $x = x_c + r \cdot \cos \theta$.
- y -posisjonen til en utøver er gitt av $y = y_c + r \cdot \sin \theta$.

x_c og y_c representerer `centerPoint` sin x - og y -koordinat, og r er radiusen.

θ er progresjonen til utøveren og er gitt av $\theta = 2\pi \cdot \frac{\text{position}}{\text{distance}}$.

Når oppgave T4 er implementert skal utøverne bevege seg rundt på kartet under simuleringen som i **Figur 11**.



Figur 11: Skjerm bilde av simuleringen etter at oppgave T4 er implementert.

Solution:

```
// TASK: T4
// Calculate the position of the skier.
TDT4102::Point Simulator::calculateSkierPosition(TDT4102::Point centerPoint,
                                                Skier &skier, int radius)
{
    // BEGIN: T4
    double theta = skier.getPosition() / map.distance * 2 * M_PI;
    return {
        centerPoint.x + static_cast<int>(std::cos(theta) * (radius)),
        centerPoint.y + static_cast<int>(std::sin(theta) * (radius))
    };
    // END: T4
}
```

5. (20 points) T5: Fyll tabellen

Vi ønsker en tabell som viser hvordan utøverne ligger an i rennet. Denne tabellen er delvis implementert, men mangler mulighet for å legge til rader. Implementer template-funksjonen `Table::addRow` i `table.h`. Denne funksjonen skal ta inn en rad i form av en vector av `TableData` som parameter. `TableData`-strukturen er deklartert i `table.h` og ser slik ut:

```
1 template <typename T>
2 struct TableData
3 {
4     string columnTitle;
5     T value;
6 };
```

Table-klassen er definert i `table.h` og har medlemsvektorer `columnTitles` og `data`, hvor henholdsvis kolonnenavn og data er lagret:

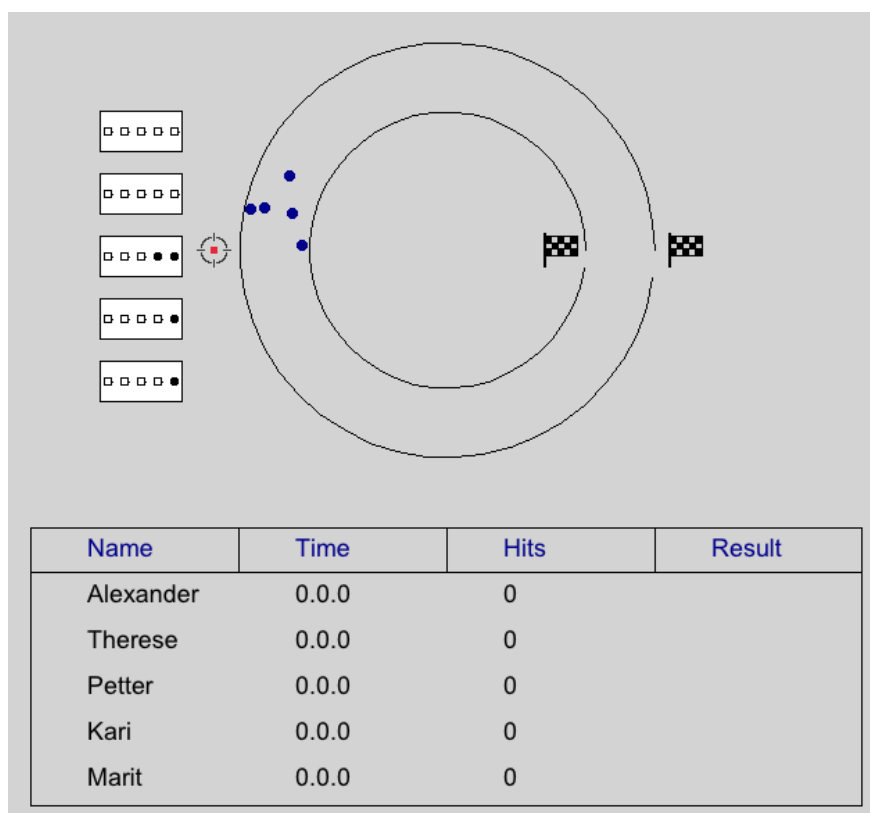
```
1 vector<string> columnTitles;
2 vector<vector<TableData<string>>> data;
```

Definer og implementer template-funksjonen `Table::addRow` i `table.h`. Funksjonen tar inn en vector av `TableData` som parameter. Funksjonen har ingen returverdi. I tillegg:

- Fjern eller kommenter ut kodelinjen `#define ADD_ROW_NOT_IMPLEMENTED` i `table.h`.
- Legg til kolonnenavnet i `Table::columnTitles`-vektoren hvis det ikke allerede eksisterer.
- Hele raden skal legges til i `Table::data`-vektoren.

Tips: Du kan legge til kodelinjen `#define ADD_ROW_NOT_IMPLEMENTED` igjen dersom du ikke har løst oppgaven enda og ønsker å fjerne feilmeldingene du får fra kompilatoren.

Når oppgave T5 er gjort skal tabellen være synlig i viduet under simuleringen slik som i **Figur 12**



Figur 12: Når oppgave T5 er gjort skal tabellen være synlig i viduet under simuleringen.

Solution:

```
// TASK: T5
// Define and implement the templated function addRow
// which checks whether the new row contains any
// new columnTitles before adding the row to
// data.
// BEGIN: T5

template<typename T>
void addRow(vector<TableData<T>> row)
{
    for (auto item : row)
    {
        bool alreadyInColumnList = false;

        for (auto columnTitle : columnTitles)
        {
            if (columnTitle == item.columnTitle)
            {
                alreadyInColumnList = true;
                break;
            }
        }

        if (!alreadyInColumnList)
        {
            columnTitles.push_back(item.columnTitle);
        }
    }

    data.push_back(row);
}

// END: T5
```

Alternative løsning ved bruk av iterasjoner:

Solution:

```
// TASK: T5
// Define and implement the templated function addRow
// which checks whether the new row contains any
// new columnTitles before adding the row to
// data.
// BEGIN: T5
```

```

template<typename T>
void addRow(vector<TableData<T>> row)
{
    for (auto item : row)
    {
        if (count(columnTitles.begin(), columnTitles.end(), item.columnTitle) < 1)
        {
            columnTitles.push_back(item.columnTitle);
        }
    }

    data.push_back(row);
}

// END: T5

```

6. (20 points) T6: Indeksering

Indeksen til en kolonne i tabellen samsvarer ikke nødvendigvis med indeksen til kolonnen i radene i data. For å finne ut hvor dataen skal plasseres, må vi derfor kunne sammenligne `TableData::columnTitle` med kolonnenavnene i `Table::columnTitles`.

Definer og implementer template-funksjonen `Table::getColumnIndexInRow` i `table.h`. Funksjonen skal ta inn en `std::string` som beskriver kolonnetittel og en vector av `TableData` som tilsvare en rad i data som parametre. Funksjonen skal returnere en `int` som tilsvare indeksen til kolonnen i raden. I tillegg

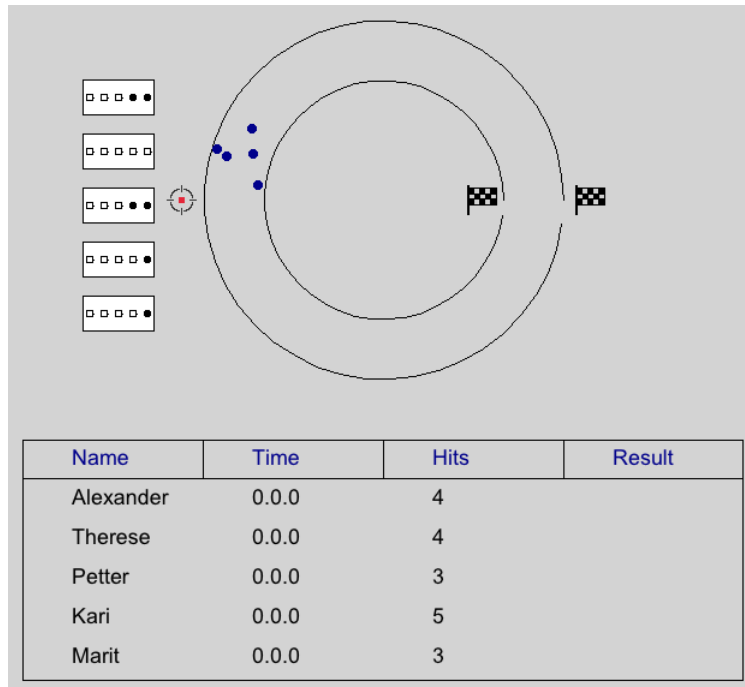
- Fjern eller kommenter ut kodelinjen

```
#define GET_INDEX_NOT_IMPLEMENTED
```

i `table.h`.
- Returner indeksen til stedet man kan finne verdien til den gitte kolonnetittelen for denne spesifikke raden.
- Dersom kolonnetittelen vi ser etter ikke finnes i data skal funksjonen returnere -1.

Tips: Du kan legge til kodelinjen `#define GET_INDEX_NOT_IMPLEMENTED` igjen dersom du ikke har løst oppgaven enda og ønsker å fjerne feilmeldingene du får fra kompilatoren.

Etter at oppgave T6 er implementert vil du kunne se at tabellen oppdaterer verdiene i *Hits*-kolonnen etter hvert som utøverne treffer blinken under skyting, som vist i [Figur 13](#)



Figur 13: Skjerm bilde av simulasjonen etter at oppgave T6 er implementert.

Solution:

```
// TASK: T6
// Define and implement the templated function getColumnIndexInRow
// which returns the index of a given columnName
// for a given row.
// BEGIN: T6

template<typename T>
int getColumnIndexInRow(string columnName, vector<TableData<T>> &row)
{
    for (std::size_t i=0; i<row.size(); i++)
    {
        if ( row[i].columnName == columnName ) {
            return static_cast<int>(i);
        }
    }

    return -1;
}

// END: T6
```

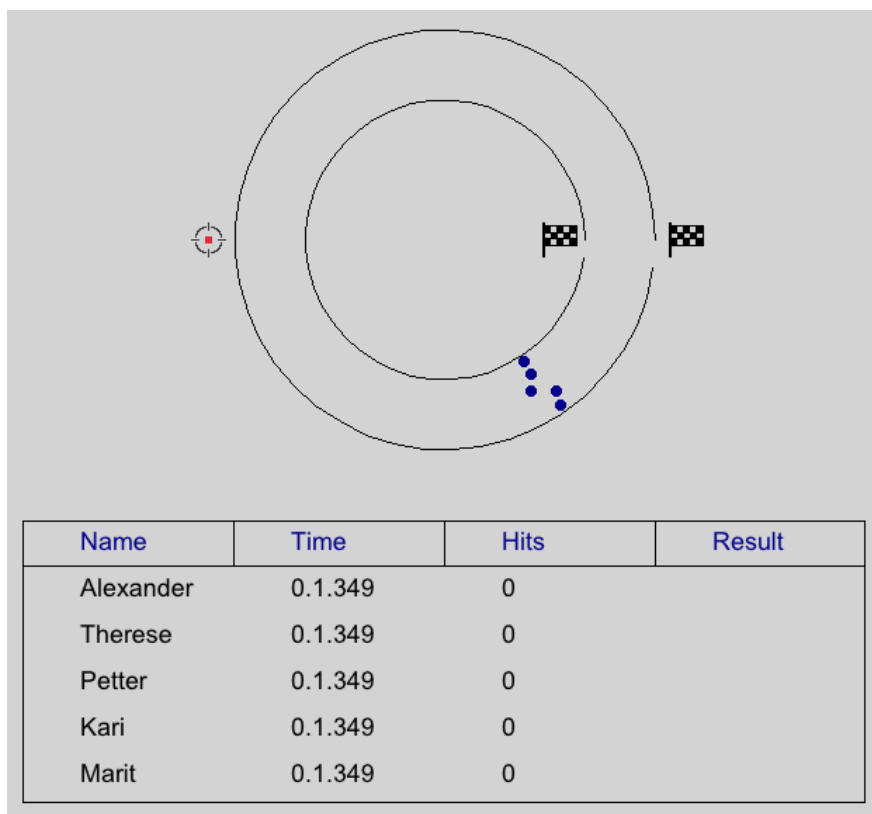
7. (15 points) **T7: Hvert sekund teller!**

Vi ønsker å registrere tidene utøverne bruker på løypa og vise det frem i tabellen i simulasjonen. Funksjonen `Stopwatch::millisecondsToString` tar inn en parameter, `timeInMilliseconds`, som er en `int` som angir hvor mange millisekunder som har gått siden Start-knappen ble trykket på. Vi ønsker å omgjøre denne tiden til et mer leselig format, litt som på en stoppeklokke.

Implementer funksjonen `Stopwatch::millisecondsToString` i `simulator.cpp`.

- Konverter verdien til `timeInMilliseconds` til antall minutter, sekunder, og millisekunder.
- Funksjonen skal returnere en `std::string` på formatet `'minutes.seconds.milliseconds'`.

Når oppgave T7 er gjort vil du kunne se at rundetidene i kolonnen *Time* oppdaterer seg under simuleringen, som vist i **Figur 14**.



Figur 14: Når oppgave T7 er gjort vil du kunne se at rundetidene i kolonnen *Time* oppdaterer seg under simuleringen.

Solution:

```
// TASK: T7
// Convert the integer timeInMilliseconds to a string on the form
```



```

// "minutes.seconds.milliseconds"
string Stopwatch::millisecondsToString(int timeInMilliseconds)
{
    // BEGIN: T7
    constexpr int MILLISECONDS_PER_MINUTE = 60000;
    constexpr int MILLISECONDS_PER_SECOND = 1000;

    int minutes = timeInMilliseconds / MILLISECONDS_PER_MINUTE;
    int remainingTime = timeInMilliseconds % MILLISECONDS_PER_MINUTE;
    int seconds = remainingTime / MILLISECONDS_PER_SECOND;
    int milliseconds = remainingTime % MILLISECONDS_PER_SECOND;

    return to_string(minutes) +
           "." + to_string(seconds) + "." + to_string(milliseconds);
    // END: T7
}

```

8. (25 points) T8: Straffetid

For å finne det endelige resultatet ønsker vi å kombinere tiden til en utøver med antall treff utøveren hadde ved å legge til straffetid. Slik tabellen er satt opp nå er tidene lagret som en `std::string`. Vi må derfor ha mulighet til å omgjøre strengen til millisekunder for å kunne legge til straffetid, og må gjøre det motsatte av det vi gjorde i forrige oppgave (T7).

Implementer funksjonen `Stopwatch::stringToMilliseconds` i `simulator.cpp`. Funksjonen tar inn en `std::string` på formatet `'minutes.seconds.milliseconds'` og skal returnere en `int` som er tiden i millisekunder.

Oppgave T8 kan ikke verifiseres visuelt.

Solution:

```
// TASK: T8
// Convert the string time, which is on the form
// "minutes.seconds.milliseconds" to an integer
// representing the time in milliseconds.
int Stopwatch::stringToMilliseconds(string time)
{
    // BEGIN: T8
    std::istringstream ss{time};
    std::array<int, 3> timeSlots = {0, 0, 0};
    std::string segment;

    std::size_t i = 0;
    while (std::getline(ss, segment, '.'))
    {
        timeSlots[i++] = std::stoi(segment);
    }

    int minutesInMilliseconds = timeSlots[0] * 60000;
    int secondsInMilliseconds = timeSlots[1] * 1000;

    int totalMilliseconds =
        minutesInMilliseconds + secondsInMilliseconds + timeSlots[2];

    return totalMilliseconds;
    // END: T8
}
```

9. (25 points) T9: Hvem havner på pallen?

Som nevnt i forrige oppgave (T8) bestemmes det endelige resultatet ved å kombinere tiden til en utøver med antall treff utøveren hadde ved å legge til straffetid. For å finne den endelige tiden til en utøver må vi dermed lese verdien som er lagret i kolonnene *Time* og *Hits*, konvertere begge verdiene fra strenger til tall, regne ut og legge til straffetid, og til slutt konvertere tiden tilbake til en streng som viser den endelige tiden på et leselig format.

Straffetiden per bom er oppgitt i den utdelte koden som konstanten `PENALTY_FACTOR`:

$$\text{PENALTY_FACTOR} = \frac{\text{PENALTY}}{\text{SPEED_SCALE}}. \quad (1)$$

Klassen `Stopwatch` inneholder funksjonene `millisecondsToString` og `stringToMilliseconds` som du implementerte i oppgave T7 og T8. Disse kan brukes til å konvertere tid frem og tilbake mellom strenger og tall. `Simulator`-klassen har en instans av `Stopwatch` kalt `watch` lagret som en `unique_ptr`. `Stopwatch`-klassen er definert i `simulator.h` og ser slik ut:

```
class Stopwatch
{
```

```

public:
    void toggleWatch();
    std::chrono::duration<int64_t, std::nano> getTime();
    string timeToString(std::chrono::duration<int64_t, std::nano> clock);
    string millisecondsToString(int time);
    int stringToMilliseconds(string time);

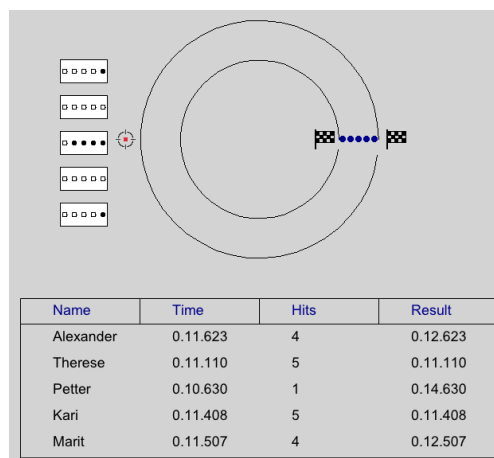
private:
    vector<std::chrono::time_point<std::chrono::steady_clock>> timeEvents;
};

```

Implementer funksjonen `Simulator::finaliseResult` i `simulator.cpp`. Funksjonen tar inn en rad fra `Table::data` og skal returnere summen av rundetiden og straffetiden som en `std::string` på formatet 'minutes:seconds:milliseconds'.

- Hent ut indeksen til kolonnen Time i raden. Dersom kolonnenavnet ikke finnes skal du returnere strengen 'Time missing'.
- Bruk Stopwatch-instansen `watch` i `Simulator` til å konvertere rundetiden til millisekunder.
- Hent ut indeksen til kolonnen Hits. Dersom kolonnenavnet ikke finnes skal du returnere strengen 'Hits missing'.
- Legg til straffetiden til rundetiden. Straffetiden per bom finnes i konstanten `PENALTY_FACTOR` (Likning (1)).
- Konverter den endelige tiden (rundetiden + straffetiden) til en `std::string` på formatet 'minutes:seconds:milliseconds'.
- Returner strengen du konstruerte i punktet over.

Når oppgave T9 er gjort vil du kunne se at tabellen oppdaterer *Result*-kolonnen etter hvert som utøverne kommer i mål, som vist i Figur 15.



Figur 15: Skjerm bilde av simulasjonen etter at oppgave T9 er gjort.

Solution:

```
// TASK: T9
// Use previously implemented functions to calculate the result of
// the skier based on time and hits.
string Simulator::finaliseResult(vector<TableData<string>> &row)
{
    // BEGIN: T9
    int index = table->getColumnIndexInRow("Time", row);

    if ( index < 0 ) {
        return "Time missing";
    }

    std::size_t index_us = static_cast<std::size_t>(index);

    int time = watch->stringToMilliseconds(row[index_us].value);

    index = table->getColumnIndexInRow("Hits", row);

    if ( index < 0 ) {
        return "Hits missing";
    }

    index_us = static_cast<std::size_t>(index);
    int hits = stoi(row[index_us].value);

    int PENALTY_FACTOR = static_cast<int>(PENALTY/SPEED_SCALE);
    int missed_shots = MAX_HITS - hits;
    int timePenalty = missed_shots * PENALTY_FACTOR;

    time += timePenalty;

    return watch->millisecondsToString(time);
    // END: T9
}
```

10. (15 points) T10: Operatoroverlasting

Vi ønsker å forenkle prosessen med å skrive ut TableData-strukturer. Vi vil derfor overlaste <<-operatoren.

Overlast <<-operatoren for TableData i table.h.

- Funksjonen skal ta inn 2 parametre, en std::ostream og en TableData-struktur med generell type.
- Vi ønsker å skrive data til strømmen på formatet 'TableData::columnTitle:TableData::value'.

Oppgave T10 kan ikke verifiseres visuelt.

Solution:

```
// TASK: T10
// Overload the << operator for the templated struct TableData.
// The form should be "title:value".
// BEGIN: T10
template <typename T>
std::ostream &operator<<(std::ostream &os, const TableData<T> &td)
{
    return os << td.columnTitle << ":" << td.value;
}
// END: T10
```

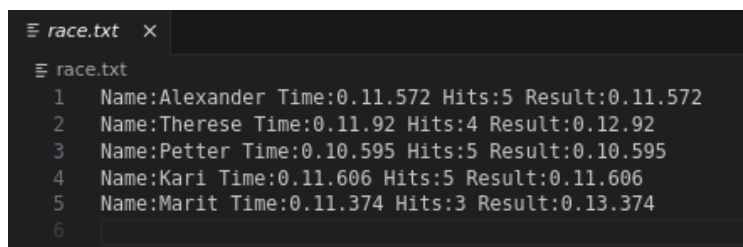
11. (20 points) T11: Lagre resultatene

Vi ønsker å lagre resultatene fra simuleringen i en fil for senere bruk.

Implementer funksjonen `Table::storeTable` i `table.cpp`.

- Dersom filepath ikke finnes skal funksjonen utløse et passende unntak.
- Data for en utøver skal lagres som en linje i filen.
- Kolonnenavnet og kolonneverdien skal lagres som `columnTitle:value` for hver kolonne, med et mellomrom mellom kolonnene. Du kan se filformatet i [Figur 16](#).

Når oppgave T11 er implementert blir det lagret en fil, `race.txt`, i hovedmappen på formatet vist i [Figur 16](#) når simuleringen er ferdig.



```
≡ race.txt x
≡ race.txt
1 Name:Alexander Time:0.11.572 Hits:5 Result:0.11.572
2 Name:Therese Time:0.11.92 Hits:4 Result:0.12.92
3 Name:Petter Time:0.10.595 Hits:5 Result:0.10.595
4 Name:Kari Time:0.11.606 Hits:5 Result:0.11.606
5 Name:Marit Time:0.11.374 Hits:3 Result:0.13.374
6
```

Figur 16: Formatet til filen som tabellen blir lagret i.

Solution:

```
// TASK: T11
// Write the Table to file.
void Table::storeTable(filesystem::path filepath)
{
    // BEGIN: T11
    std::ofstream fs{filepath};

    if (!fs.is_open())
    {
        std::string errorMessage =
            "Unable to open file for writing at path: " + filepath.string();
        throw std::runtime_error(errorMessage);
    }

    for (const auto &row : data)
    {
        for (const auto &cell : row)
        {
            fs << cell << " ";
        }
        fs << std::endl;
    }

    fs.close();
    // END: T11
}
```